AquaGuard IQ

A Data-Driven Filter-Life Monitoring and Predictive Maintenance for Water Purifiers

Track 2 — Water Purifier | Area: Prognostics of filter failure using data analytics

Team Name: ProjectX	Institute: Amrita Vishwa Vidyapeetham
Team Member: Beemireddy Venkata Bharghava Reddy	Department: Artificial Intelligence and Data Science (AI&DS)

A. Team and Institutional Background

AquaGuard IQ is a low-cost prototype designed for monitoring the use of filters on domestic water purifiers and for aiding in the planning of maintenance. It achieves this by combining embedded sensing technology, data analytics, and a Python-based dashboard so that users will find it easier to understand both filter usage and their maintenance requirements. The system makes use of components that are easily obtained and open-source software, which means it is appropriate for use by students. This makes it suitable for a student-built prototype on a limited budget.



B. Problem and Proposed Idea

Problem Statement

Since water purifier filters need to be serviced or replaced from time to-time, people often rely just on set time intervals rather than on the actual amount of water used. It is difficult to know when maintenance should take place because there is no clear information regarding the total amount of water used and the changing readings. A simple and inexpensive monitoring system could increase awareness of usage and give earlier reminders for maintenance.

Proposed Solution

AquaGuard IQ will use an inline water-flow sensor,ESP32 microcontroller, plus a Python dashboard that would compute the total quantity of water used and present an estimate for filter maintenance. The system will monitor actual drinking-water usage with the stated filter usage due to the OEM specification plus time passed since the last maintenance. In places where equipment and calibration are appropriate, TDS trends may be recorded as supporting evidence but not as a proof of filter failure or water quality. Historical data will be evaluated using a transparent baseline estimator while machine-learning methods will be used once sufficient labeled filter-life data has been collected.

Baseline and Target Performance

The prototype targets the following performance levels: (1) water-volume estimation within ±10% of reference measurements across repeated comparative tests, and (2) at least 95% flow-event detection under specified test conditions. These are engineering targets and have not yet been experimentally verified. Prediction accuracy will be reported after evaluation against relevant reference data.

Technical Distinction

The distinguishing feature is the combination of cumulative water-volume tracking, manufacturer-rated service limits, service history, historical usage trends and a clear user dashboard in a low-cost design. The initial version is a usage-based maintenance estimator, not a proven machine-learning filter-failure predictor.

C. Technical Feasibility

An ESP32 development board along with a compatible water-flow sensor and a Python Streamlit application will be utilized to create the prototype. The water flow generates sensor pulsations, which the ESP32 counts while calculating the amount of volume with the help of a calibration factor determined by experiment. A cumulative readout will be sent to the computer over USB where the time and date will be stamped and the results stored in Python. The evaluator function will check if cumulative use equals the filter

manufacturer water filter capacity and was within user-defined service time. Initially, independent bench testing will be performed instead of modifying an existing water purifier. The electronic components will be isolated from water, all the wet connections will be inspected for seepage, and low-voltage USB supply will be used. The system will provide advice on the maintenance and will not provide an assessment for the safety of drinking water nor ensure detection of every filter failure.

D. Execution Feasibility

Preliminary Components

- ESP32 development board
- Compatible inline water-flow sensor
- Breadboard, jumper wires and suitable connectors
- USB data cable and suitable power source
- Sample containers and measuring jug for calibration
- Computer running Python, Streamlit, Pandas and Plotly

System-Level Block Diagram

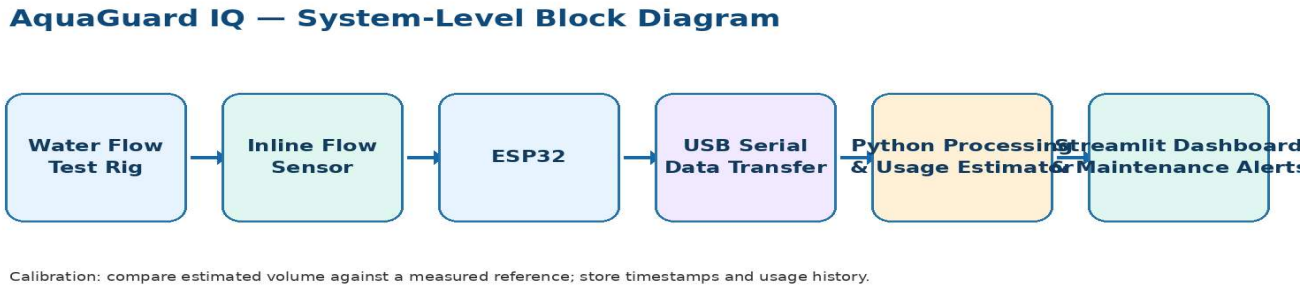


Figure 1. Proposed system architecture.

Prototype Concept (Illustrative)

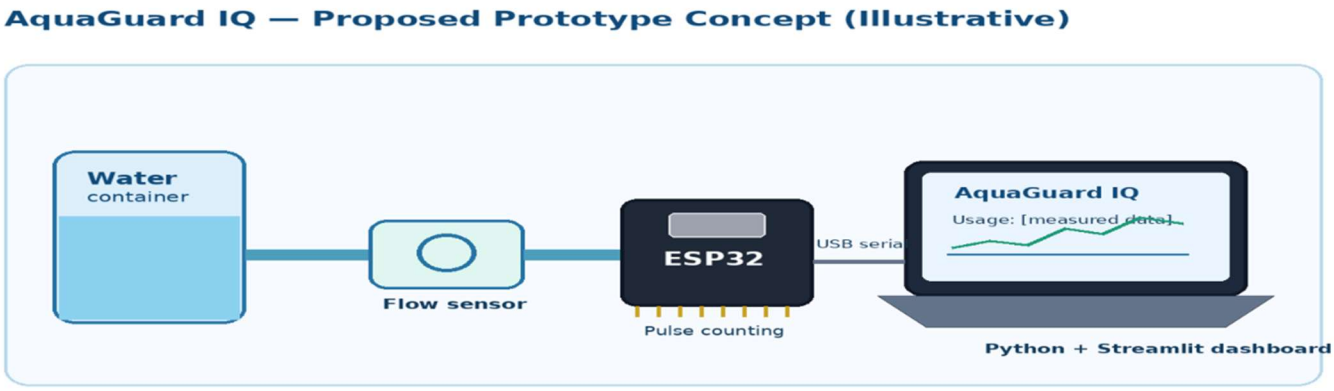


Figure 2. Concept illustration only

Preliminary Budget

Component / Item	Estimated Cost (₹)
ESP32 development board	350–500
Water-flow sensor	200–350
Wiring and assembly	200–350
Connectors and testing materials	200–400
Indicative subtotal	**950–1,600**
Contingency / miscellaneous buffer	100–400
Target total	**1,050–2,000**

These are preliminary estimates, not supplier quotations. Confirm local prices and shipping before procurement; avoid double-counting items already available in the team.

Component Sourcing Risks

Flow sensors vary by connector type, flow-rate range, pulse output and calibration requirements. A compatible model must be selected before finalising the circuit. No custom fabrication or imported components are planned for the first prototype. Measurement accuracy may vary with flow rate, sensor mounting and water conditions.

Build Plan to Round 2

1. Assemble an independent flow-sensor test rig and check pulse counting using measured water volumes.
2. Send cumulative usage readings from the ESP32 to a Python application over USB serial.
3. Build a Streamlit dashboard showing cumulative volume, usage trends and a preliminary maintenance estimate.
4. Run repeated trials; calculate volume-estimation error and flow-event detection rate under defined conditions.
5. Document the setup, results, limitations, actual costs and future plan for collecting labelled filter-life data.

Testing and Validation Flowchart

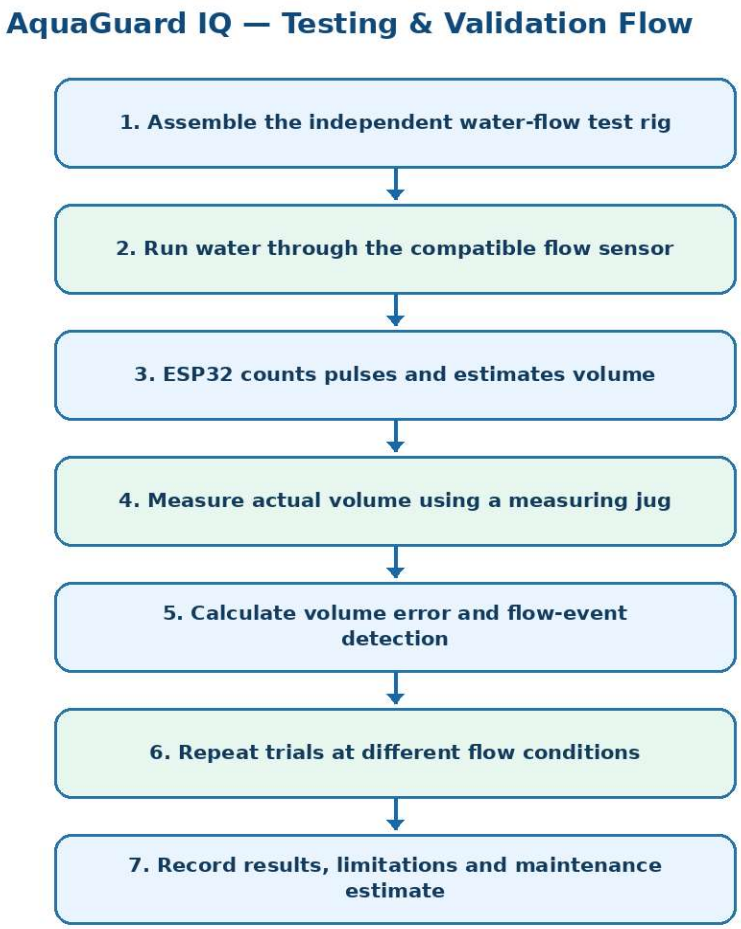


Figure 3. Planned test sequence.

Expected Outcome

AquaGuard IQ aims to demonstrate a practical, inexpensive way to track filter usage and translate water-consumption measurements into understandable maintenance guidance. The prototype will establish a foundation for future validation using real filter-performance and replacement-cycle data. Any claims about filter-failure prediction will depend on the quality and quantity of that future data.